

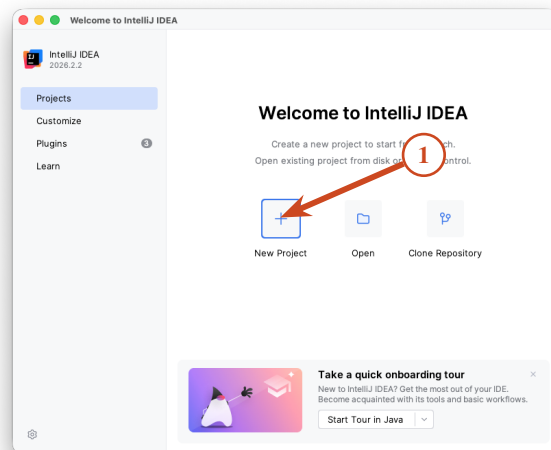
Algorithmes et Pensée Computationnelle

Premiers pas avec Python dans IntelliJ IDEA

La semaine dernière, vous avez appris à créer et à exécuter des programmes depuis la ligne de commande, notamment en leur passant des arguments. Cette semaine, vous allez voir comment réaliser ces mêmes tâches directement dans IntelliJ IDEA.

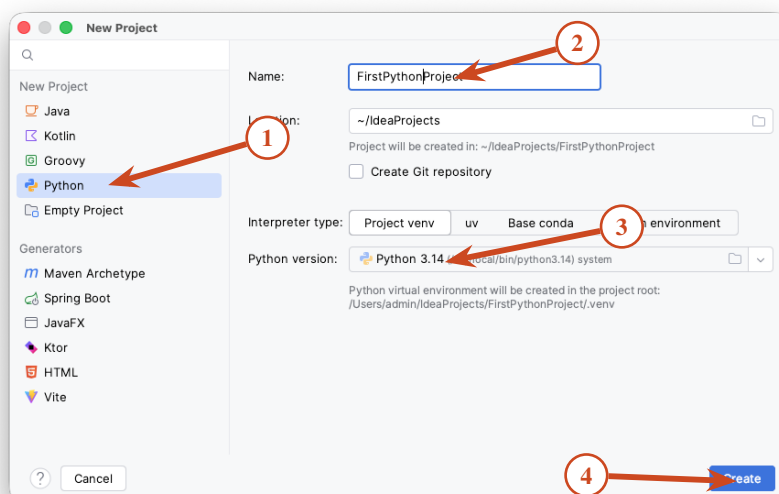
1 Créer un projet Python dans IntelliJ IDEA

Ouvrez IntelliJ IDEA. Sur l'écran d'accueil, sélectionnez **New Project**.



L'écran d'accueil d'IntelliJ IDEA.

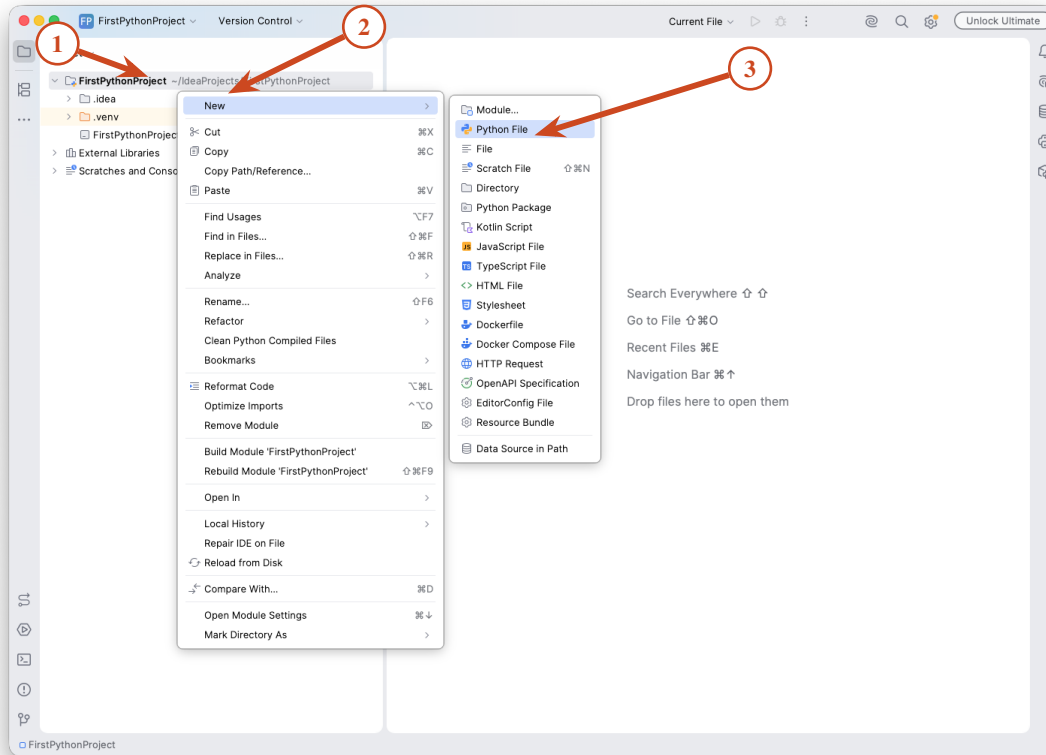
Dans la fenêtre de création du projet, choisissez **Python** (1), indiquez un nom et un emplacement pour le projet (2), puis sélectionnez un interpréteur Python (3). La capture montre un environnement virtuel propre au projet. Votre version de Python et vos chemins d'accès peuvent être différents. Sélectionnez **Create** (4).



Choisissez Python, nommez le projet, vérifiez l'interpréteur et créez le projet.

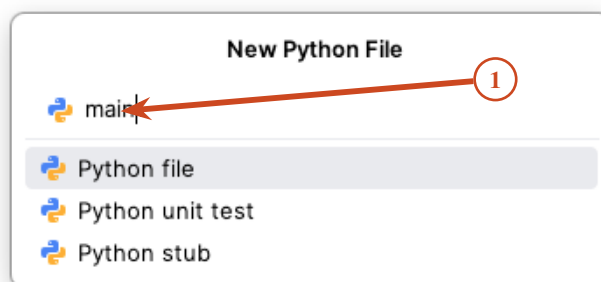
Créer un fichier Python

Dans le panneau **Project**, faites un clic droit sur le nom du projet (1), choisissez **New** (2), puis **Python File** (3). Les deux menus ci-dessous se superposent à la fenêtre du projet, comme dans IntelliJ IDEA.



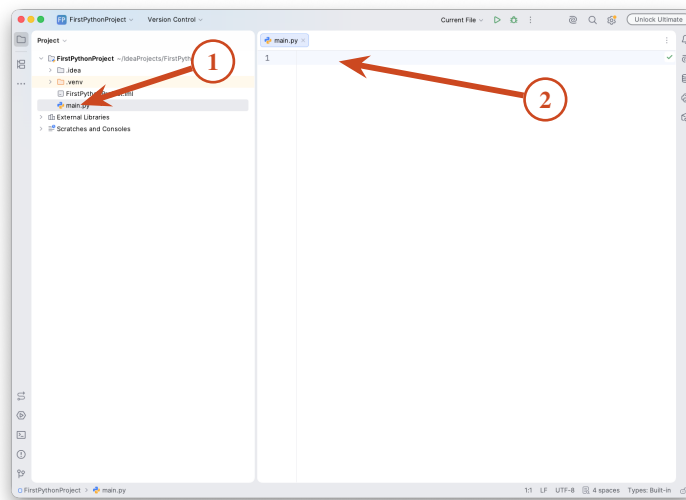
La fenêtre du projet et les deux menus contextuels dans leur ordre d'apparition.

Saisissez `main` comme nom de fichier, laissez **Python file** sélectionné, puis appuyez sur Entrée. IntelliJ ajoute l'extension `.py`.



Nommez le nouveau fichier `main.py`.

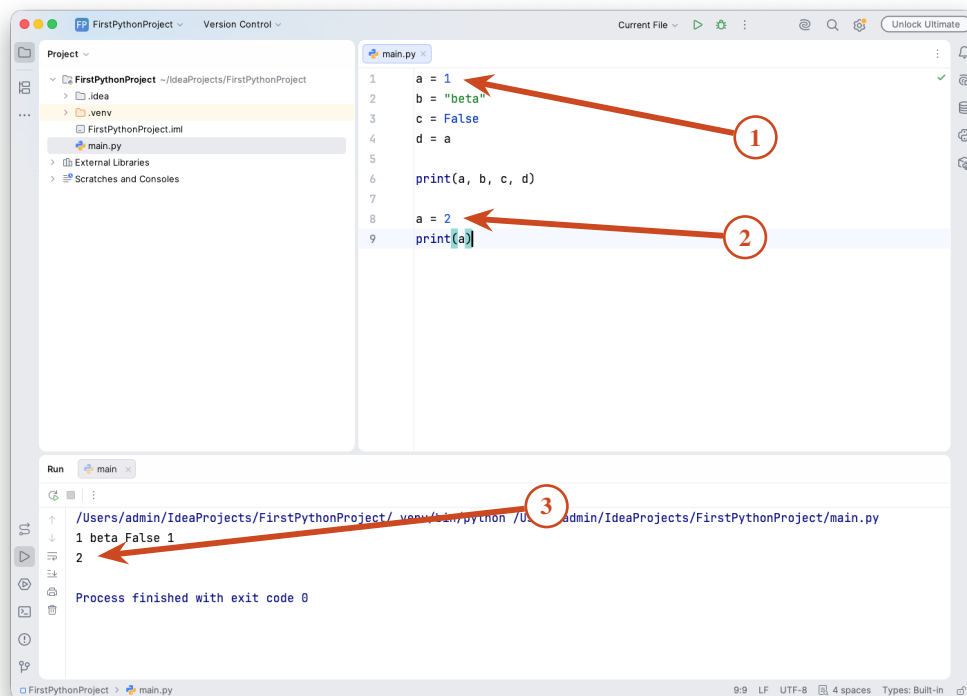
Le fichier apparaît maintenant dans le panneau **Project** (1), et son éditeur vide s'ouvre à droite (2). Cliquez dans l'éditeur pour saisir du code Python.



Le nouveau fichier dans le panneau du projet et sa zone d'édition.

2 Créer des variables

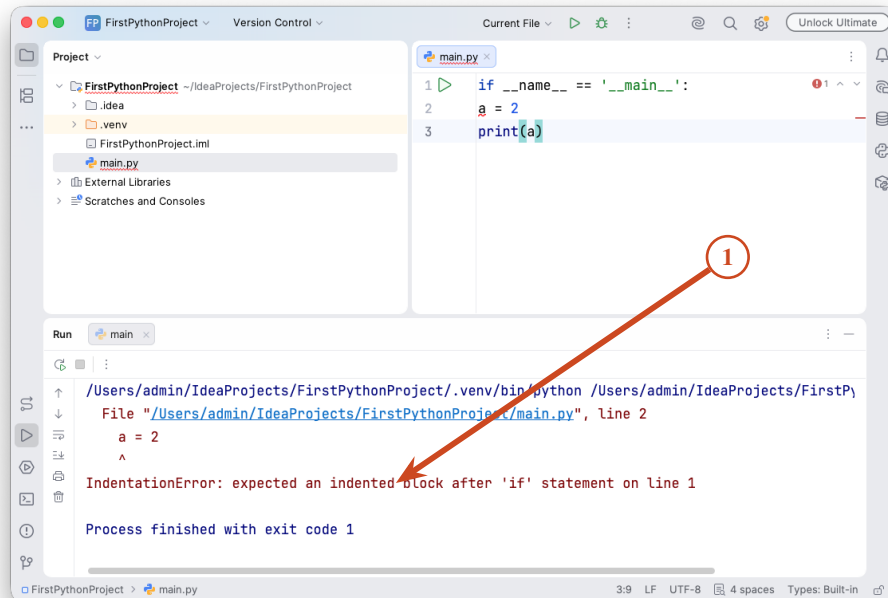
Une variable associe un nom à une valeur. En Python, on attribue une valeur avec `=` : `a = 1` stocke un nombre, `b = "beta"` stocke du texte et `c = False` stocke une valeur booléenne. Il n'est pas nécessaire de déclarer le type à l'avance. Dans la capture, `d = a` copie la valeur actuelle de `a`. Après `a = 2`, `d` vaut toujours 1. Les deux lignes du panneau **Run** montrent cette différence.



Les affectations (1), la réaffectation (2) et les résultats affichés (3).

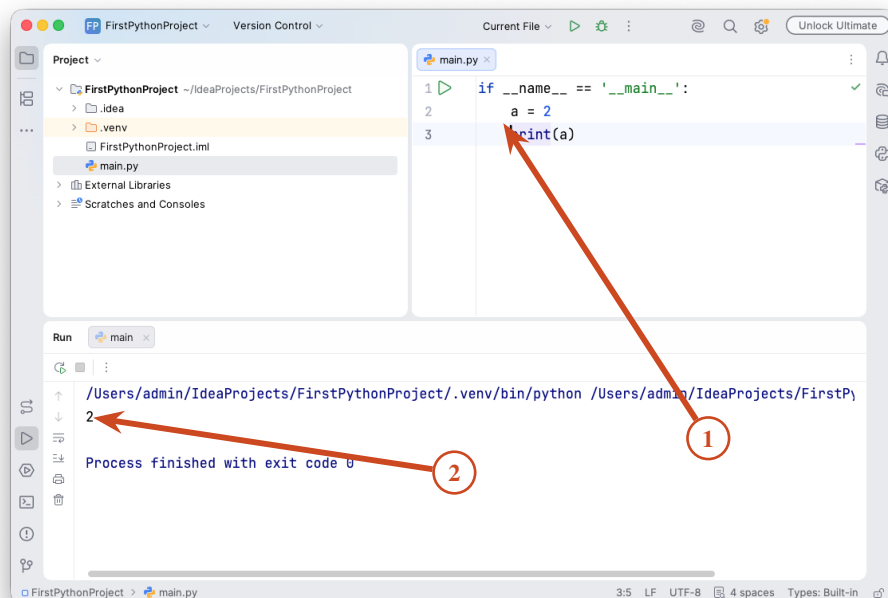
3 L'importance de l'indentation

En Python, l'indentation regroupe les instructions en blocs. Une ligne terminée par deux-points, comme la ligne `if` ci-dessous, doit être suivie d'un bloc indenté. Dans la première capture, `a = 2` reste aligné sur la marge gauche : Python signale donc une erreur `IndentationError` (flèche 1).



Aucun bloc indenté ne suit la condition : le programme s'arrête avec une erreur.

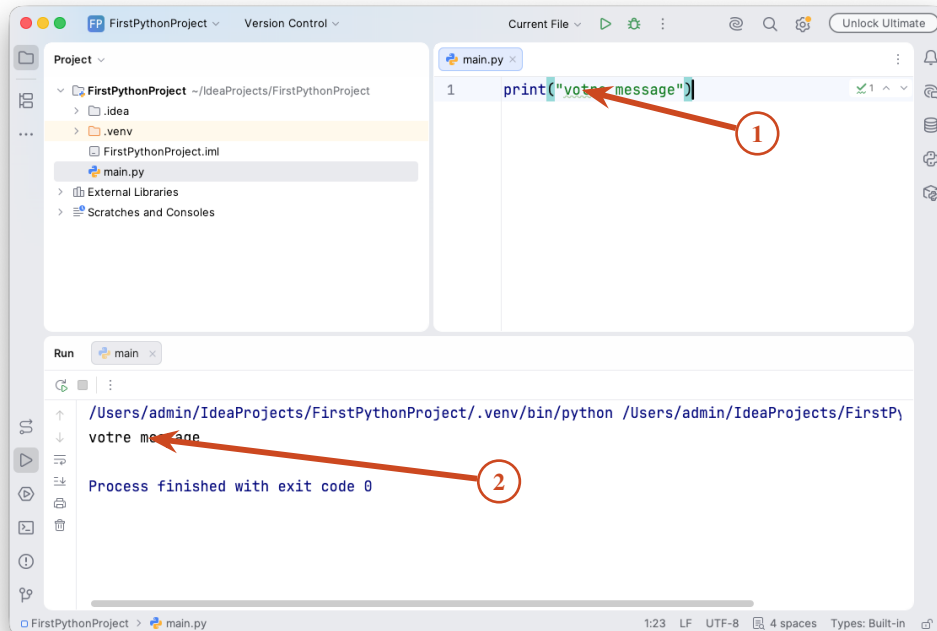
Indentez les deux instructions sous la ligne `if`, comme dans la version corrigée (flèche 1). Quatre espaces constituent un niveau d'indentation courant. Le panneau **Run** affiche maintenant 2 (flèche 2). Les instructions d'un même bloc doivent être alignées au même niveau.



Les deux instructions sont indentées : le programme s'exécute et affiche 2.

4 Les fonctions

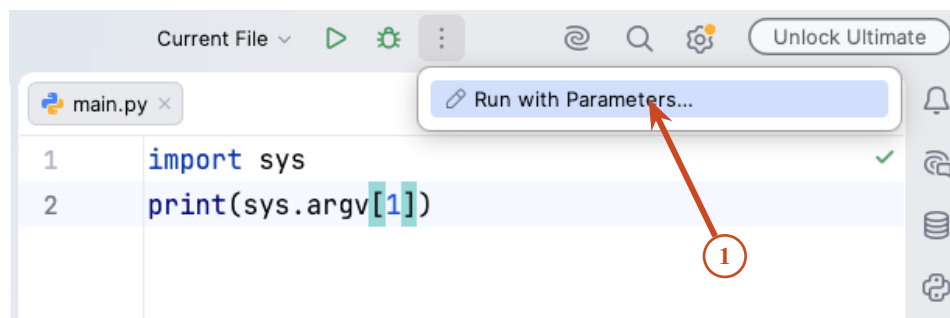
`print()` est une fonction Python. Elle affiche dans le panneau **Run** la valeur placée entre ses parenthèses. Saisissez `print("votre message")` (flèche 1), puis cliquez sur le bouton vert d'exécution en haut de la fenêtre. Le résultat est `votre message` (flèche 2) : les guillemets délimitent le texte dans le code, mais ne sont pas affichés.



L'appel de la fonction `print()` dans l'éditeur et son résultat dans le panneau d'exécution.

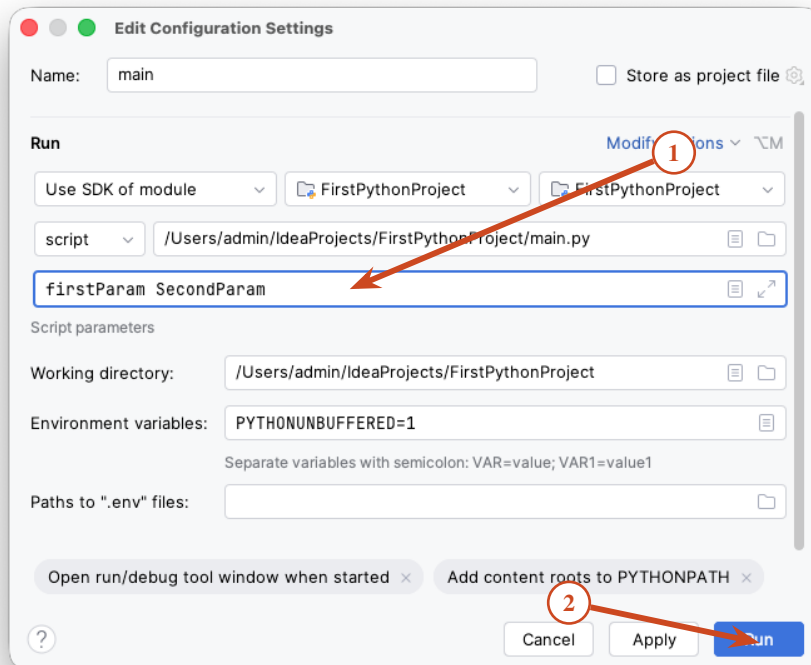
5 Les arguments de la ligne de commande

Les arguments de la ligne de commande sont des valeurs transmises au lancement d'un programme. Dans `main.py`, saisissez `import sys`, puis `print(sys.argv[1])`. Ici, `sys.argv[1]` désigne le premier argument fourni par l'utilisateur; `sys.argv[0]` contient le nom du programme. Ouvrez le menu à trois points à côté des commandes d'exécution, puis choisissez **Run with Parameters...** (flèche 1).



La commande de lancement avec paramètres et le programme qui lit un argument.

Dans **Script parameters**, saisissez `firstParam SecondParam` (flèche 1), puis sélectionnez **Run** (flèche 2). IntelliJ transmet les deux mots comme deux arguments distincts. Le programme affiche `firstParam`, car il utilise `sys.argv[1]`; `sys.argv[2]` désignerait `SecondParam`. Fournissez au moins un argument avant d'exécuter ce programme.



Saisissez les paramètres du programme, puis lancez son exécution.